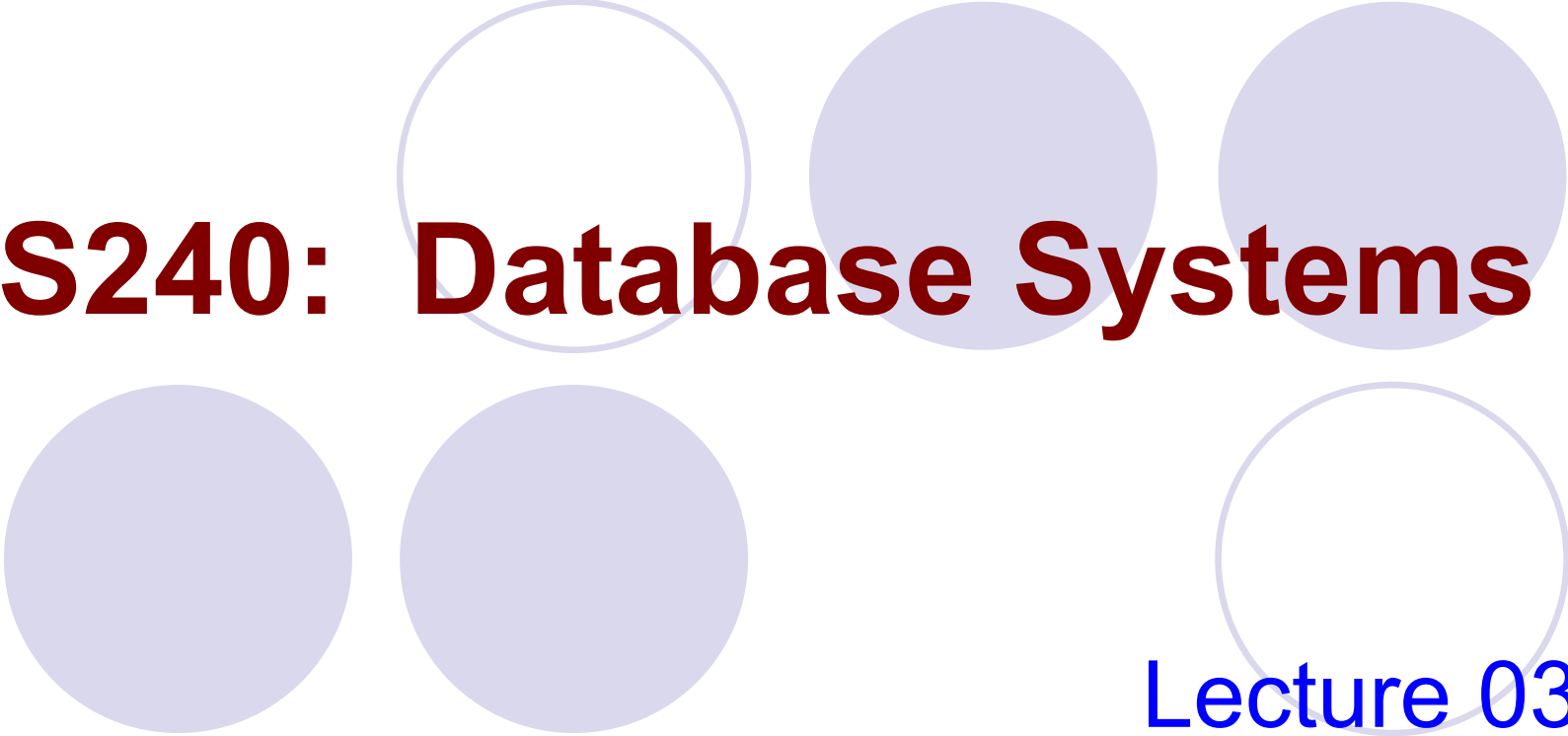




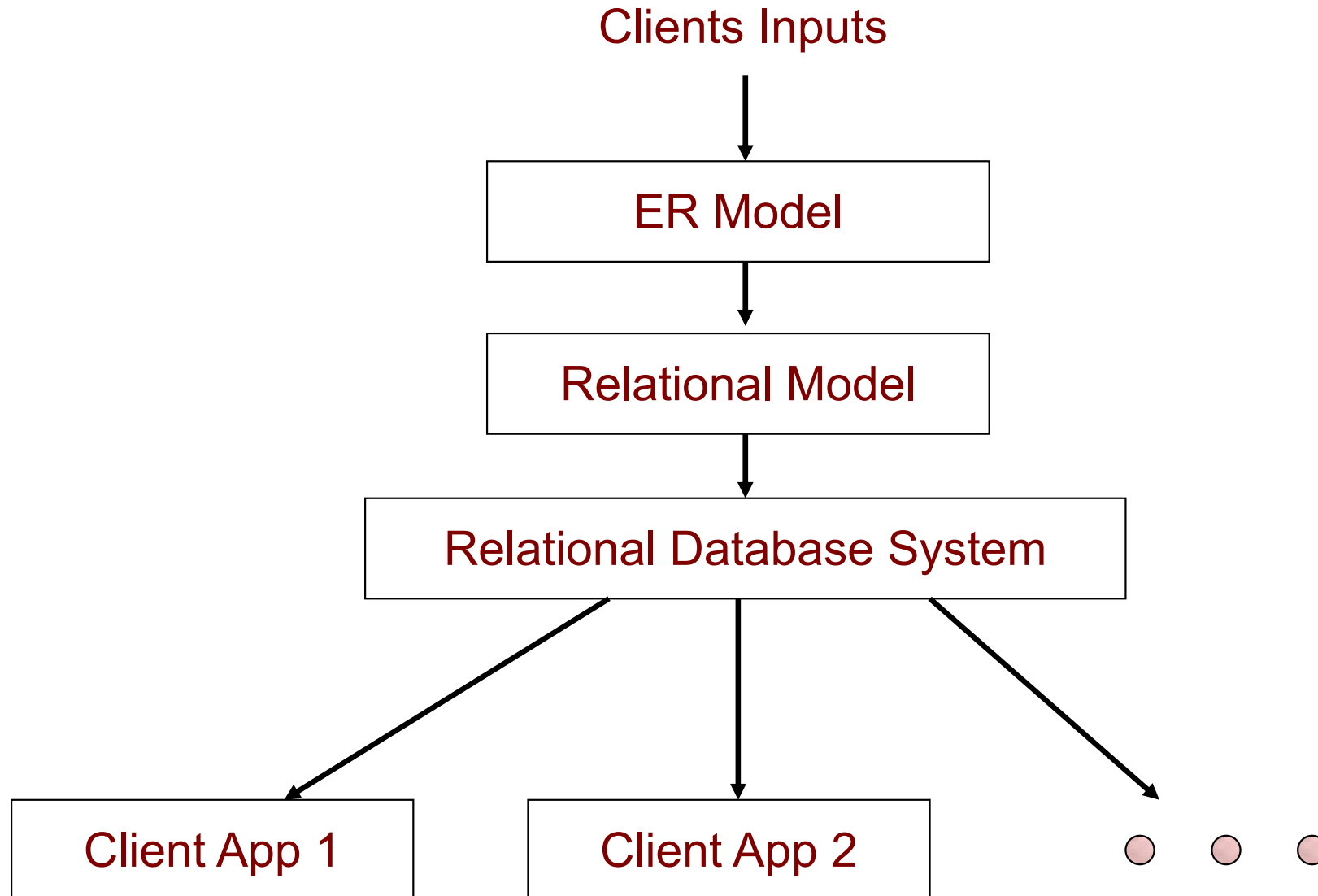
CS240: Database Systems

Lecture 03: Relational Model

Outline

- **Relational Model**
 - **Relational Model Concepts**
 - Relational Database Schemas
- ER-to-Relational Mapping
 - Translating traditional ER diagrams
 - Translating Class Hierarchy

Database Modeling



Introduction

- The relational model was first introduced by Ted Codd of IBM Research in 1970
- Attracted due to its simplicity, elegance and mathematical foundations
- The model uses the concept of a mathematical relation – which looks like a **table** of values
- The **SQL Query Language** was developed by IBM in the 1970's

Why Study the Relational Model?

- The relational model is by far the dominant data model
- It is the foundation for the leading DBMS products of many vendors such as Oracle, Microsoft SQL Server, etc.
- A major strength of the relational model is that it supports simple, powerful querying of data
- Queries can be written intuitively, and the DBMS can perform efficient evaluation

The Relational Data Model

- The relational Model of Data is based on the concept of a *Relation*, a Relation is a mathematical concept based on the ideas of sets
- Represents data as a collection of relations
- **Table** of values
 - Row
 - Represents a collection of related data values
 - Fact that typically corresponds to a real-world entity or relationship
 - Table name and column names
 - Interpret the meaning of the values in each row *attribute*

Relation Name/Table Name

Attributes/Columns (collectively as a schema)

STUDENT				
Name	<u>Student-id</u>	Age	GPA	
Dave	99223367	19	3.3	Tuples/Rows
Jones	96882145	18	3.4	
Smith	96452165	18	3.2	
Mary	96154292	20	3.5	
Tom	96520934	22	3.8	

- **Relation** $\leftrightarrow$ **table**; denoted by $R(A_1, A_2, \dots, A_n)$ where R is a **relation name** and $(A_1, A_2, \dots, A_n)$ is the **relation schema** of R
- **Attribute (column)** $\leftrightarrow$ denoted by A_i
 - Dom(Age): [0-100]
 - Dom(EmpName): 50 alphabetic chars
 - Dom(Salary): non-negative integer
- **Tuple (Record)** $\leftrightarrow$ row
- **Attribute value** $\leftrightarrow$ value stored in a table cell
- **Domain** $\leftrightarrow$ legal type and range of values of an attribute, denoted by $\text{dom}(A_i)$

Relation Schema

- The Schema is a description of a Relation:
 - Denoted by $R(A_1, A_2, \dots, A_n)$, R is the name of the relation
 - The attributes of the relation are $A_1, A_2, \dots, A_n$
 - All legal values of an attribute is called its domain
- Example:
- $\text{STUDENT}(\text{Name}, \text{Student-id}, \text{Age}, \text{GPA})$
 - STUDENT is the relation name
 - Defined over the four attributes: Name, Student-id, Age, CGA
 - A STUDENT state may include 5 STUDENTs; another 250 STUDENTs

Relations States are Sets

- Relation State r , or $r(R)$: a specific state of relation R – this is a set of tuples (rows)
 - $r(R) = \{t_1, t_2, \dots, t_n\}$ where each t_i is an n -tuple, $t_i = \langle v_1, v_2, \dots, v_n \rangle$ where each v_j element-of $\text{dom}(A_j)$
 - **Mathematical relation** of degree n on the domains $\text{dom}(A_1), \text{dom}(A_2), \dots, \text{dom}(A_n)$
 - **Subset** of the **Cartesian product** of the domains that define R : $r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$
- All tuples in a relation state $r(R)$ form a set
- By definition, there cannot be duplicates in a set
- By definition, set elements (tuples) are **not ordered**, even though tuples frequently appear to be in the tabular form

Example

- Let $R(A_1, A_2)$ be a relation schema:
 - Let $\text{dom}(A_1) = \{ 0, 1 \}$
 - Let $\text{dom}(A_2) = \{ a, b, c \}$
- Then: $\text{dom}(A_1) \times \text{dom}(A_2)$ is all possible combinations: $\{ \langle 0, a \rangle, \langle 0, b \rangle, \langle 0, c \rangle, \langle 1, a \rangle, \langle 1, b \rangle, \langle 1, c \rangle \}$
- The relation state $r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2)$
- For example: $r(R)$ could be $\{ \langle 0, a \rangle, \langle 0, b \rangle, \langle 1, c \rangle \}$
 - this is one possible state r of the relation R , defined over A_1 and A_2
 - it has three 2-tuples: $\langle 0, a \rangle, \langle 0, b \rangle, \langle 1, c \rangle$

Values in Relations

- We refer to **component values** of a tuple t by:
 - $t[A_i]$ or $t.A_i$
 - This is the value v_i of attribute A_i for tuple t
- All values are considered *atomic* (indivisible)
- Each value in a tuple must be from the domain of the attribute for that column, i.e., $v_i = t[A_i] \in \text{dom}(A_i)$
- Allow a special **NULL** value is used to represent values that are *unknown* or *inapplicable* to certain tuples

Values in Tuple

- All values are considered *atomic* (indivisible):
 - Flat relational model
 - Composite and multivalued attributes not allowed
 - First normal form assumption
 - Multivalued attributes
 - Must be represented by separate relations
 - Composite attributes
 - Represented only by simple component attributes in basic relational model

Values in Tuple

<u>studentNo</u>	studentName	sex	birthday	age	address			classNo
					province	city	street	

Fig.1 Composite attributes

<u>studentNo</u>	studentName	sex	birthday	age	province	city	street	classNo
------------------	-------------	-----	----------	-----	----------	------	--------	---------

Fig. 2 Simple component attributes

NULL Values

- **NULL** in a tuple:
 - Represent the values of attributes that may be unknown or may not apply to a tuple
 - Meanings for **NULL** values
 - Value unknown
 - Value exists but is not available
- IMPORTANT: **NULL** $\neq$ **NULL**

Schema Definition in SQL

- The relation schema is

Customer(customer-name, customer-street, customer-city)

or

Customer	<u>customer-name</u>	customer-street	customer-city
----------	----------------------	-----------------	---------------

- The primary key is underlined in the above

```
CREATE TABLE Customer
(
    customer-name    CHAR(20)    NOT NULL,
    customer-street  CHAR(30),
    customer-city    CHAR(30),
    PRIMARY KEY (customer-name)
)
```

Schema Definition in SQL

- The SQL standard supports a variety of built-in types, including:
 - int(4B), smallint(2B), tinyint(1B);
 - float, real(4B), decimal(p, n), numeric(p, n);
 - char(n), varchar(n), text;
 - binary(n), varbinary(n), image;
 - bit;
 - money(8B), small money(4B);
 - datetime(4B, since 1753.1.1), smalldatetime(4B, since 1900.1.1)

Schema Definition in SQL

- To remove a relation from an SQL database, we use the **drop table** command:

drop table r

- We use the alter table command to add or delete attributes to an existing relation
- All records in the relation are assigned null for a new attribute.

alter table customer **add** phone char(10)

alter table customer **drop** phone

Schema Definition in SQL

- Tuples are inserted using the **INSERT** command.
- We can insert a single tuple into the Student table as follows:

```
INSERT INTO Student (Name, Student-id, Age, GPA)  
VALUES ( 'Judy', 99223368, 21, 3.9)
```

- Tuples are deleted using the **DELETE** command.
- We can delete all students tuples with name equal to Smith as follows:

```
DELETE  
FROM Student S  
WHERE S.name='Smith'
```

Schema Definition in SQL

- We can modify the column values in an existing row using the **UPDATE** command.
- We can increment the age and decrement the gpa of the student with student-id **99223367** as follows:

```
UPDATE Student S  
SET S.age=S.age+1, S.gpa=S.gpa-1  
WHERE S.student-id=99223367
```

Summary

Informal Terms	Formal Terms
Table	Relation
Column Header	Attribute
All possible Column Values	Domain
Row	Tuple
Table Definition	Schema of a Relation
Populated Table	State of the Relation

Outline

- **Relational Model**
 - Relational Model Concepts
 - **Relational Database Schemas**
- ER-to-Relational Mapping
 - Translating traditional ER diagrams
 - Translating Class Hierarchy

Relational Databases

- **Relational database schema S**

- Set of relation schemas $S = \{R_1, R_2, \dots, R_m\}$
- Set of integrity constraints (ICs)

- **Relational database state**

- Set of relation states $DB = \{r_1, r_2, \dots, r_m\}$
- Each r_i is a state of R_i and such that the r_i relation states satisfy integrity constraints specified in IC
- **Invalid state**: Does not obey all the integrity constraints ICs
- **Valid state**: Satisfies all the constraints in the defined set of integrity constraints ICs

Relational Integrity Constraints (IC)

- Constraints are **conditions** that must hold on **all** valid relation states.
- There are some *main types* of constraints in the relational model:
 - **Domain** constraint
 - Every value in a tuple must be from the *domain of its attribute* (or it could be **NULL**, if allowed for that attribute)
 - **Key** constraints
 - **Entity integrity** constraints
 - **Referential integrity** constraints
- ICs are specified when the schema is defined
- ICs are checked when relations are modified

Domain Constraints

- The value of an attribute is limited to its domain.
- A domain can impose rules on both formats and valid value ranges.
 - A salary value cannot be negative
 - An employee's name cannot be **NULL**
 - This is called the **NOT NULL** constraint

- Example:

Constraint name (optional): if the constraint is violated, the constraint name is returned and can be used to identify the error

```
CREATE DOMAIN hourly-wage NUMERIC(5,2)
CONSTRAINT wage-value-test CHECK (value > 4.00)
```

- The domain hourly-wage is a decimal number with 5 digits, 2 of which are placed after the decimal point
- The domain has a constraint that ensures that the hourly wage is greater than 4.00

Key Constraints

- Certain minimal subset of the fields (candidate key) of a relation is a unique identifier for a record
- Out of all the available candidate keys, a database designer can identify a primary key
 - The DBMS may create an index with the primary key
 - The primary key attributes are underlined
- The primary key is used to *reference* the tuple from another tuple
 - General rule:
 - choose as primary key the smallest of the candidate keys (in terms of space)
 - choice is sometimes subjective

Key in SQL

- In SQL, we can declare
 - a key by the **UNIQUE** command
 - a primary key by the **PRIMARY KEY** constraint

```
CREATE TABLE Students
```

```
(
```

```
    sid    CHAR(20),
```

```
    name   CHAR(30),
```

```
    login  CHAR(20),
```

```
    age    INTEGER,
```

```
    gpa    REAL,
```

```
    address CHAR(30),
```

```
    UNIQUE (name, address),
```

```
    CONSTRAINT StudentsKey PRIMARY KEY (sid)
```

```
)
```

Primary key

key

Constraint name (optional)

Entity Integrity

- The *primary key attributes* PK of each relation schema R in S **CANNOT** have **NULL** values in any tuple of $r(R)$
 - This is because primary key values are used to *identify* the individual tuples
 - $t[PK] \neq \mathbf{NULL}$ for any tuple t in $r(R)$
 - If PK has several attributes, **NULL** is not allowed in any of these attributes

No primary key value can be **NULL**

Referential Integrity

- Referential Integrity (RR) constraint specified between two relations.
Maintains consistency among tuples in two relations
- A **foreign key** is a set of attributes in one relation $R1$ that is used to refer to a tuple in another relation $R2$
- Tuples in the **referencing relation** $R1$ have attributes FK (called **foreign key** attributes) that reference the **primary key** attributes PK of the **referenced relation** $R2$
 - A tuple $t1$ in $R1$ is said to **reference** a tuple $t2$ in $R2$ if $t1[FK] = t2[PK]$
- A RR constraint can be displayed in a relational database schema as a directed arc  **FROM** $R1.FK$ **TO** $R2.PK$.

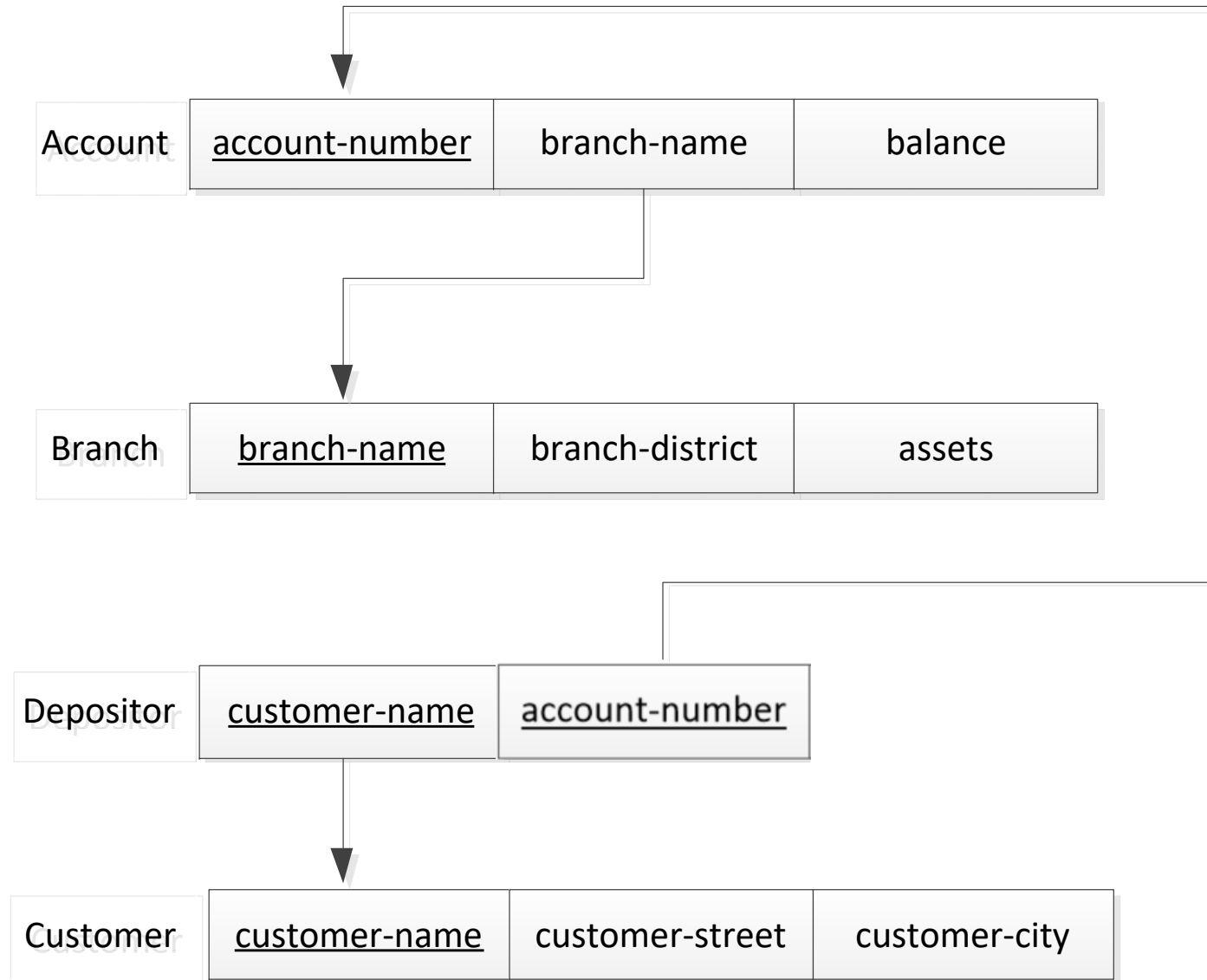
Referential Integrity

- Statement of the constraint
 - The value in FK of the **referencing relation** R1 can be **either**:
 - (1) a value of an existing primary key value of a corresponding primary key PK in the **referenced relation** R2, or
 - (2) a **NULL**
 - In case (2), the FK in R1 should **NOT** be a part of its own primary key (because of Entity Integrity Constraint)

Example 1: Relational Schema Diagram

- Student(Student-id, Student-Name)
- Take(Student-id, Course-id, grade)
- Course(Course-id, Course-Name)
- (Student-id, Course-id) in relation Take is a primary key
- **foreign keys?**
- Student-id, course-id in relation Take are foreign keys
- Draw a relational schema diagram specifying the foreign keys for this schema

Example 2: Foreign Key



Example 2: Foreign Key

Account

<u>account-number</u>	branch-name	balance
-----------------------	-------------	---------

```
CREATE TABLE account
```

```
(
```

```
  account-number    CHAR(10)  NOT NULL,
```

```
  branch-name       CHAR(15),
```

```
  balance           INTEGER,
```

```
  PRIMARY KEY (account-number)
```

```
  FOREIGN KEY (branch-name) REFERENCES branch
```

```
)
```

Branch

<u>branch-name</u>	branch-district	assets
--------------------	-----------------	--------

Example 2: Foreign Key

Depositor

customer-name

account-number

```
CREATE TABLE depositor
```

```
customer-name    CHAR(20)  NOT NULL,
```

```
account-number   CHAR(10)  NOT NULL,
```

```
PRIMARY KEY (customer-name, account-number),
```

```
FOREIGN KEY (customer-name) REFERENCES customer
```

```
FOREIGN KEY (account-number) REFERENCES account
```

Account

account-number

branch-name

balance

Customer

customer-name

customer-street

customer-city

Populated Database State

- Each *relation* will have many tuples in its current relation state
- The **relational database state** is a union of all the individual relation states
- Whenever the database is changed, a new state arises
- Basic operations for changing the database:
 - INSERT a new tuple in a relation
 - DELETE an existing tuple from a relation
 - MODIFY an attribute of an existing tuple

Possible Violations for INSERT

- Domain constraint:
 - if one of the attribute values provided for the new tuple is not of the specified attribute domain
- Key constraint:
 - if the value of a key attribute in the new tuple already exists in another tuple in the relation
- Referential integrity:
 - if a foreign key value in the new tuple references a primary key value that does not exist in the referenced relation
- Entity integrity:
 - if the primary key value is null in the new tuple

Example:

Account(account-number, branch-name, balance)

Branch(branch-name, branch-district, assets)

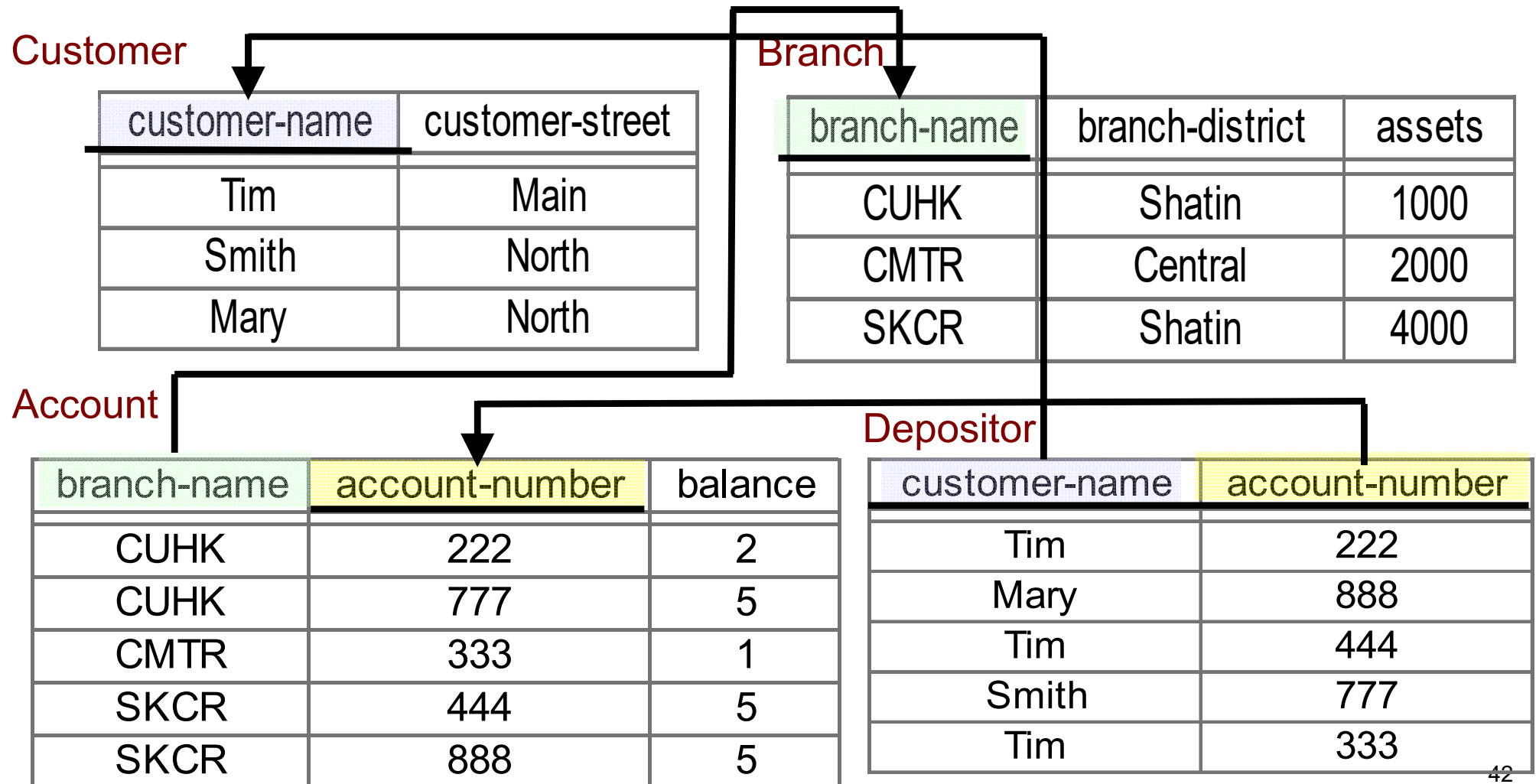
Depositor(customer-name, account-number)

Customer(customer-name, customer-street, customer-city)

cannot add (KLN, 111, 3) to Account

cannot add (Chris, 222) to Depositor

cannot add (Mary, 999) to Depositor



Possible Violations for DELETE

- Referential constraints
 - If the primary key value of the tuple being deleted is referenced from other tuples in the database
- Options of remedies:
 - **RESTRICT**: reject the deletion
 - **CASCADE**: delete the record in the referencing table
 - **SET NULL**: set the foreign keys of the referencing tuples to NULL
 - **SET DEFAULT**: set the foreign keys of the referencing tuples to default value

Example:

Account(account-number, branch-name, balance)

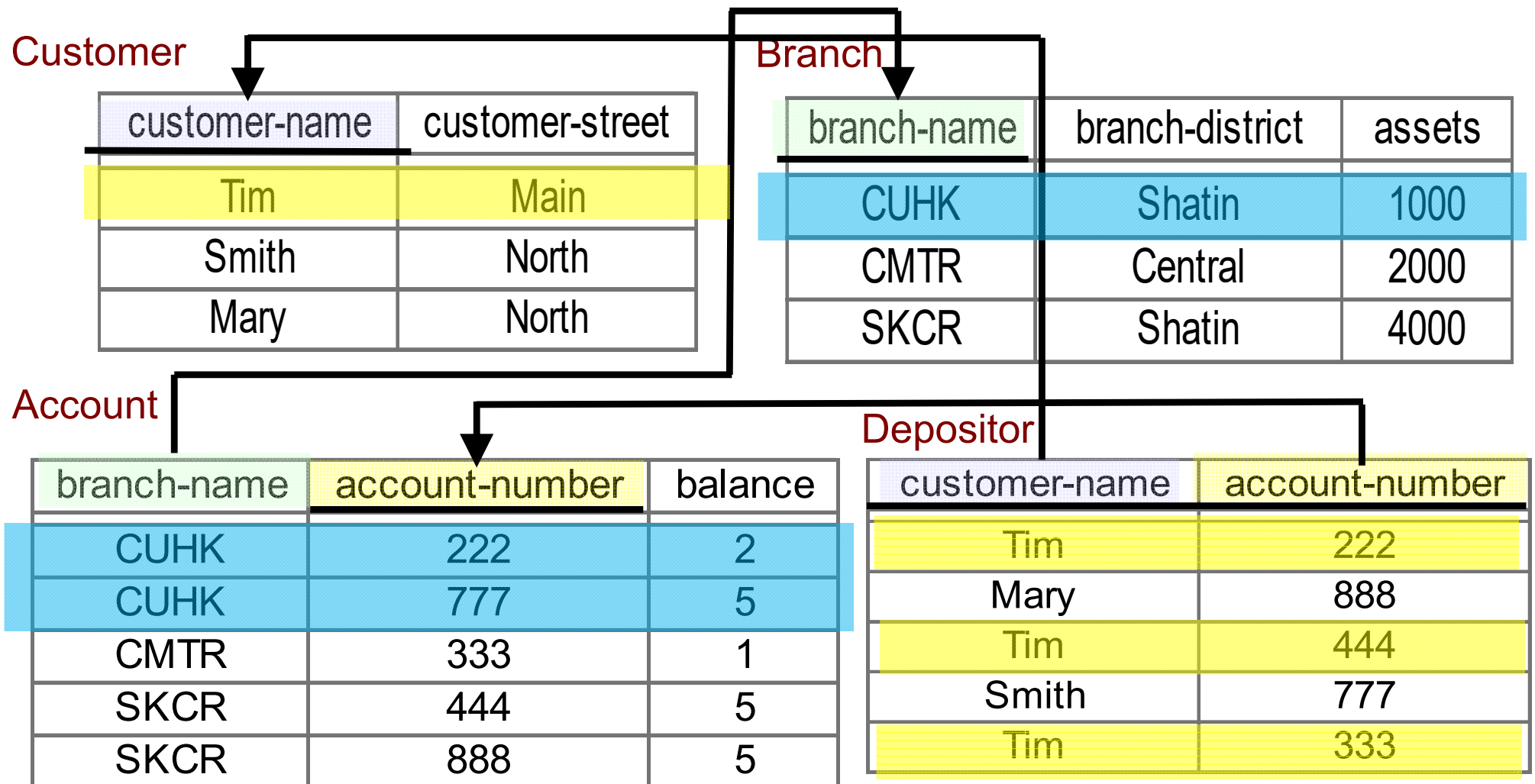
Branch(branch-name, branch-district, assets)

Depositor(customer-name, account-number)

Customer(customer-name, customer-street, customer-city)

Cannot simply delete Customer Tim

Cannot simply delete branch CUHK



Example: Foreign Key

- Cascading delete:
 - Delete Customer Tim, cascaded delete on depositor
 - Delete branch CUHK, cascaded delete on account

```
CREATE TABLE account
(
    branch-name      CHAR(15),
    account-number   CHAR(10)    NOT NULL,
    balance          INTEGER,
    PRIMARY KEY (account-number),
    FOREIGN KEY (branch-name) REFERENCES branch
                        ON DELETE CASCADE
                        ON UPDATE CASCADE
                        [ON UPDATE NO ACTION]
)
```

Possible Violations for UPDATE

- Constraint violations depending on the attribute being updated:
 - Updating the primary key (PK):
 - Similar to a DELETE followed by an INSERT
 - Need to specify similar options to DELETE
 - Updating a foreign key (FK):
 - May violate referential integrity
 - Updating an ordinary attribute (neither PK nor FK):
 - Can only violate domain and business rules constraints